NetKAT: Fondamenti Semantici per le Reti

POPL '14

Mario Rossi

Università di Torino

1. Introduzione e Motivazioni

Il contesto: Reti vs. Computing

Computing moderno:

- Virtualizzazione, OS standard, astrazioni potenti.
- Innovazione rapida via software.

Networking tradizionale:

- "L'ultimo bastione del mainframe".
- Black boxes.
- Configurazione manuale error-prone.

Rivoluzione: Software Defined Networking

- Control plane vs. Data Plane.
- Standard OpenFlow.
- Linguaggi sopra OpenFlow.

Il problema:

- OpenFlow è Assembly.
- I linguaggi esistenti (Frenetic, etc.) erano *ad-hoc*.
- **Manca framework formale, minimale, stabile**.

2. Costruire NetKAT

Di cosa abbiamo bisogno?

1. **Logica (Il Controllo):** Biforcare condizionalmente il comportamento.
2. **Composizione (Il Flusso):** Insieme di percorsi complessi.
3. **Stato (I Dati):** Definire concretamente cos'è un "pacchetto".

La risposta di NetKAT:

$$\text{NetKAT} = \underbrace{\text{Packet Algebra}}_{\substack{\text{Stato} \\ \text{(Campi, Modifiche, Storia)}}} + \overbrace{\underbrace{\text{KA}}_{\substack{\text{Composizione} \\ \text{(Seq, Loop, Scelta)}}} + \underbrace{\text{T}}_{\substack{\text{Logica} \\ \text{(If-Then-Else)}}}}^{\text{KAT}}$$

Ripasso: Kleene Algebra (KA)

$(K, +, \cdot, *, 0, 1)$ tale che:

- $(K, +, \cdot, 0, 1)$ semianello idempotente.
- $p^* \cdot q = \min\{r \mid p \cdot r + q \leq r\} \quad (p \cdot q^* = \min\{r \mid r \cdot p + q \leq r\})$.

- **Scelta ($+$):** $p + q$.
- **Sequenza ($\cdot$):** $p \cdot q$.
- **Iterazione ($*$):** p^*.

Standard per cammini su grafi e automi.

Ripasso: Kleene Algebra with Tests (KAT)

Algebra estesa a due sorte $(K, B, +, \cdot, *, 0, 1, \neg)$:

- K (Espressioni): Gli elementi che rappresentano azioni o trasformazioni.
- $B \subseteq K$ (Test): Forma un'algebra booleana.

Interazione:

- Algebra booleana immersa dentro a algebra di Kleene.
- Test come filtri componibili.
- ($\neg$ applicabile solo a B).

Il dominio dei pacchetti

L'algebra opera sulla struttura dei pacchetti.

$$pk ::= \{f_1 = v_1, f_2 = v_2, ..., f_n = v_n\}$$

Le operazioni atomiche di NetKAT si dividono in:

1. **Test ($f = n$):**
   - Se $pk.f = n \rightarrow$ lascia passare il pacchetto.
   - Se $pk.f \neq n \rightarrow$ scarta il pacchetto.
2. **Modifiche ($f \leftarrow n$):**
   - Aggiornano il campo f al valore n.
   - Il pacchetto modificato prosegue nella sequenza.

Il modello "Logical Crossbar"

Come rappresentiamo l'intera rete in una formula?

Ingredienti:

- **Policy (p):** Logica degli switch (es. $sw = A \cdot \dots + sw = B \cdot \dots$).
- **Topologia (t):** Anche i link sono espressioni (es. $t = sw = A \cdot pt = 1 \cdot sw \leftarrow B \cdot pt \leftarrow 2 + \dots$)

Esecuzione globale:

$$p_{net} \triangleq \text{in} \cdot (p \cdot t)^* \cdot \text{out}$$

3. Semantica e Meta-Teoria

Semantica con packet histories

Stato (H): Lista non vuota $h = pk_0 :: pk_1 :: \cdots :: pk_n$.

Funzione Semantica $[\![p]\!] : H \to \mathcal{P}(H)$

- **Atomici**: $[\![f \leftarrow n]\!](pk :: h) = \{pk[f := n] :: h\}$, $[\![\text{dup}]\!](pk :: h) = \{pk :: pk :: h\}$
- **Unione**: $[\![p + q]\!](h) = [\![p]\!](h) \cup [\![q]\!](h)$
- **Sequenza (Composizione di Kleisli)**:

$$[\![p \cdot q]\!](h) = ([\![p]\!] \bullet [\![q]\!])(h) = \bigcup_{h' \in [\![p]\!](h)} [\![q]\!](h')$$

- **Iterazione**:

$$[\![p^*]\!](h) = \bigcup_{i \in \mathbb{N}} [\![p^i]\!](h)$$

Teoria equazionale

1. **Assiomi KA & KAT:** (es. $p + p = p$, $b \cdot p + \neg b \cdot p = p$).
2. **Packet Axioms:** Regole specifiche per manipolare i campi.

Esempi di Packet Axioms

- $f \leftarrow n \cdot f' \leftarrow m \equiv f' \leftarrow m \cdot f \leftarrow n$ (Commutatività, se $f \neq f'$).
- $f \leftarrow n \cdot f = n \equiv f \leftarrow n$ (Ridondanza del test dopo scrittura).
- $\mathrm{dup} \cdot f = n \equiv f = n \cdot \mathrm{dup}$ (I test commutano con dup).

Meta-teoria

Soundness:

$$\vdash p \equiv q \implies [\![p]\!] = [\![q]\!]$$

Gli assiomi non mentono rispetto alla semantica.

Completeness:

$$[\![p]\!] = [\![q]\!] \implies \vdash p \equiv q$$

L'uguaglianza semantica è sempre dimostrabile algebricamente.

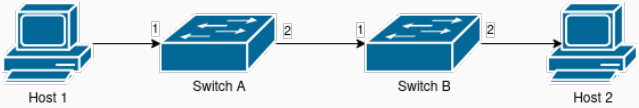
Decidability:

Dimostrazione equivalenza PSPACE-complete.

4. Applicazioni e Verifica

Esempio: Access control (SSH)

Scenario: Host A invia a Host B. Bloccare traffico SSH.



- $p_A \triangleq (sw = A \cdot \neg(typ = SSH) \cdot p) + (sw = B \cdot p)$
- $p_B \triangleq (sw = A \cdot p) + (sw = B \cdot \neg(typ = SSH) \cdot p)$

Obiettivo:

$$\text{in} \cdot (p_A \cdot t)^* \cdot \text{out} \stackrel{?}{\equiv} \text{in} \cdot (p_B \cdot t)^* \cdot \text{out}$$

Reachability

"Può A comunicare con B?"

Definizione: b è raggiungibile da a se esiste una traccia

$$\langle pk_1, \ldots, pk_n \rangle \in \mathrm{rng}(\llbracket \mathrm{dup} \cdot (p \cdot t \cdot \mathrm{dup})^* \rrbracket)$$

tale che $\llbracket a \rrbracket(pk_n) = \{pk_n\}$ e $\llbracket b \rrbracket(pk_1) = \{pk_1\}$.

Verifica equazionale:

$$a \cdot \mathrm{dup} \cdot (p \cdot t \cdot \mathrm{dup})^* \cdot b \not\equiv 0$$

Waypointing

"Tutto il traffico passa per il firewall w?"

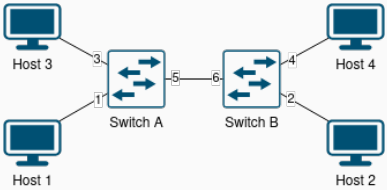
Definizione: w è un waypoint da a a b se, per tutte le storie $\langle pk_1, \ldots, pk_n \rangle \in \mathrm{rng}([\![\mathrm{dup} \cdot (p \cdot t \cdot \mathrm{dup})^*]\!])$, se la storia inizia in a e finisce in b, allora esiste un pacchetto pk_x nella storia tale che $[\![w]\!](pk_x) = \{pk_x\}$.

Verifica equazionale:

$$\underbrace{a \cdot \mathrm{dup} \cdot (p \cdot t \cdot \mathrm{dup})^* \cdot b}_{\text{Totale}} \leq \underbrace{a \cdot \mathrm{dup} \cdot (\neg b \cdot p \cdot t \cdot \mathrm{dup})^* \cdot w \cdot (\neg a \cdot p \cdot t \cdot \mathrm{dup})^* \cdot b}_{\text{Passante per W}}$$

Traffic isolation: idea

Scenario: Due sviluppatori configurano la stessa rete per due scopi diversi.



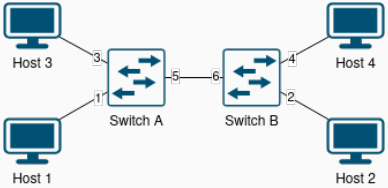
$$p_{B1} \triangleq sw = B \cdot (pt = 6 \cdot pt \leftarrow 2 + pt = 2 \cdot pt \leftarrow 6)$$
$$p_{B2} \triangleq sw = B \cdot (pt = 6 \cdot pt \leftarrow 4 + pt = 4 \cdot pt \leftarrow 6)$$

Idea: Comporre policy indipendenti tramite unione $(+)$

$$p_{net} \overset{?}{\triangleq} (((p_{A1} + p_{B1}) + (p_{A2} + p_{B2})) \cdot t)^*$$

Traffic isolation: problema

Scenario: Due sviluppatori configurano la stessa rete per due scopi diversi.



$$p_{B1} \triangleq sw = B \cdot (pt = 6 \cdot pt \leftarrow 2 + pt = 2 \cdot pt \leftarrow 6)$$
$$p_{B2} \triangleq sw = B \cdot (pt = 6 \cdot pt \leftarrow 4 + pt = 4 \cdot pt \leftarrow 6)$$

Idea: Comporre policy indipendenti tramite unione $(+)$

$$p_{net} \stackrel{?}{\triangleq} (((p_{A1} + p_{B1}) + (p_{A2} + p_{B2})) \cdot t)^*$$

Problema: Duplicazione di pacchetti.

Traffic Isolation: soluzione

Network slice:

$$s = \{in\}\ w : (p)\ \{out\}$$

Espansione

$$\begin{aligned} \text{let } \mathbf{pre} &= (\text{tag} = w_0 \cdot \text{in} \cdot \mathbf{tag} \leftarrow w) + (\text{tag} = w) \\ \text{let } \mathbf{post} &= (\text{out} \cdot \mathbf{tag} \leftarrow w_0) + (\neg \text{out}) \\ \text{in} &\quad (\mathbf{pre} \cdot p \cdot \mathbf{post}) \end{aligned}$$

5. Compilazione

Processo di compilazione

L'input del compilatore è una policy NetKAT "ristretta":

- Niente dup (la history non esiste nello switch hardware).
- Niente $sw \leftarrow n$ (lo spostamento è fisico, non logico).

Processo di quattro passi:

1. **Star elimination:** Rimuove i cicli (*).
2. **Switch specialization:** Isola la logica per ogni nodo.
3. **Conversione in ONF:** Genera regole prioritarie.
4. **Combining results:** Genera tabella OpenFlow.

Step 1: Star elimination

Lemma 9

$$p \in \mathsf{NetKAT}^{-(\mathsf{dup},\mathsf{sw}\leftarrow)} \implies \exists p' \in \mathsf{NetKAT}^{-(\mathsf{dup},\mathsf{sw}\leftarrow,*)} \text{ t.c. } p \equiv p'$$

Semplificazione:

$$p \xrightarrow{\text{Normalizzazione}} R(p) \subseteq A \cdot \cancel{(\Pi \cdot \mathsf{dup})^*} \cdot \Pi \implies p \equiv \sum_{\mathbf{i}} (\alpha_{\mathbf{i}} \cdot \pi_{\mathbf{i}})$$

Step 2: Switch specialization

Come otteniamo le policy locali dalla policy globale?

Idea: Per ogni switch sw_i,

$$p_i \triangleq p[1/sw = sw_i][0/sw = sw_j] \mid j \neq i$$

Esempio:

$$p \triangleq sw = A \cdot port \leftarrow 3 + sw = B \cdot port \leftarrow 4$$
$$p_A \triangleq 1 \cdot port \leftarrow 3 + 0 \cdot port \leftarrow 4 \equiv port \leftarrow 3$$
$$p_B \triangleq 0 \cdot port \leftarrow 3 + 1 \cdot port \leftarrow 4 \equiv port \leftarrow 4$$

Step 3: Conversione in OpenFlow Normal Form

Tabella prioritaria $\iff \ell ::= \text{if } b \text{ then } as \text{ else } \ell \stackrel{\text{def}}{=} (p \cdot as) + (\neg p \cdot \ell)$

Prima	**Dopo**
$f_1 = v_1 \cdot \cdots \cdot f_n = v_n$	Immutato
$f_1 \leftarrow v_1 \cdot \cdots \cdot f_n \leftarrow v_n$	Immutato
$p + q$	Prodotto incrociato
$p \cdot q$	Commutazione modifiche-test

Union

$$
\begin{aligned}
&(\text{if dst} = A \text{ then pt} := 1 \text{ else } 0) + \\
&(\text{if src} = B \text{ then pt} := 2 \text{ else } 0) \\
=\ &\text{if dst} = A \cdot \text{src} = B \text{ then pt} := 1 + \text{pt} := 2 \\
&\text{else if dst} = A \text{ then pt} := 1 \\
&\text{else if src} = B \text{ then pt} := 2 \\
&\text{else } 0
\end{aligned}
$$

Sequence

$$
\begin{aligned}
&(\text{if typ} = \text{SSH then vlan} := W \text{ else } 1) \cdot \\
&(\text{if dst} = A \text{ then pt} := 1 \ldots \text{ else } 0) \\
=\ &\text{if dst} = A \cdot \text{typ} = \text{SSH then vlan} := W \cdot \text{pt} := 1 \\
&\text{else if dst} = A \text{ then pt} := 1 \\
&\text{else if dst} = B \cdot \text{typ} = \text{SSH then } \ldots \\
&\text{else if dst} = B \text{ then pt} := 2 \\
&\text{else } 0
\end{aligned}
$$

$$\textbf{ONF} ::= (sw = sw_1 \cdot \ell_1) + \cdots + (sw = sw_n \cdot \ell_n)$$

Conclusioni

NetKAT ad oggi

1. Ricerca accademica attiva.
2. Applicazioni industriali limitate.
3. Pioniere dei linguaggi formali per reti.
4. Influenza su framework più applicativi (e.g. P4)

Grazie.